

CS100433

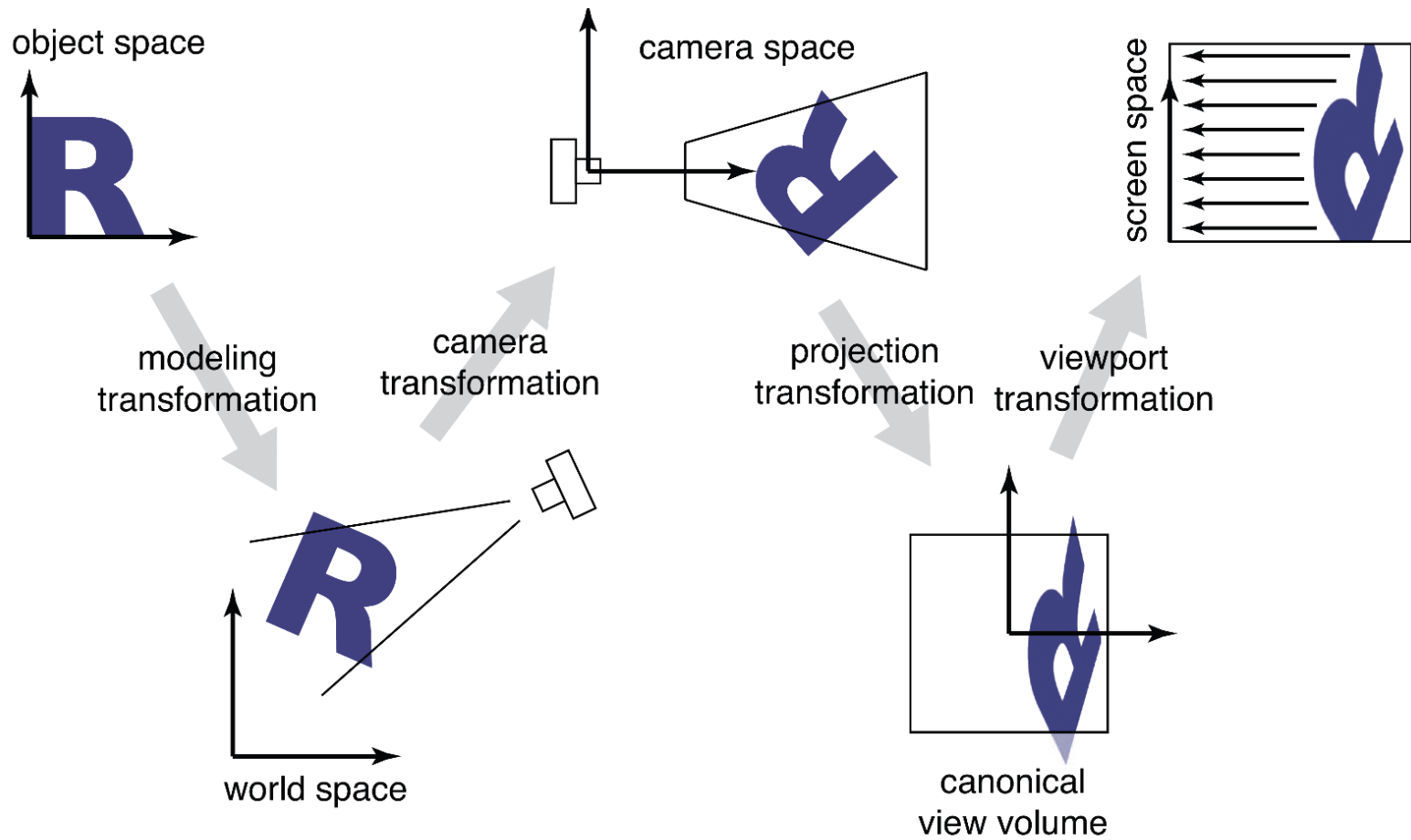
2D and 3D Transformation

Junqiao Zhao 赵君峤

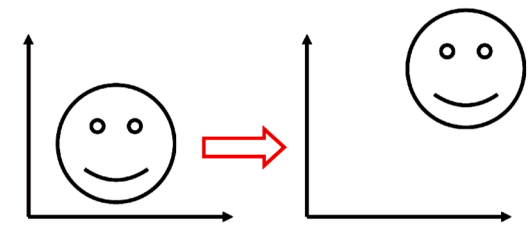
School of Computer Science and Technology

Tongji University

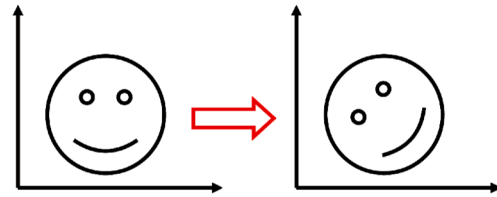
Why transformation?



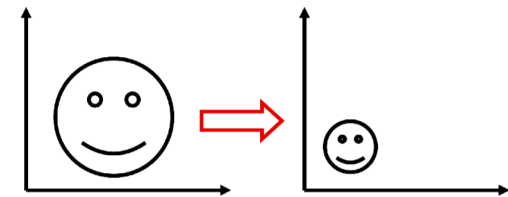
Transformations



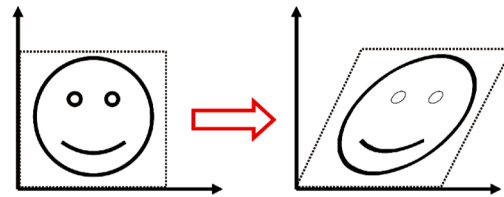
translation



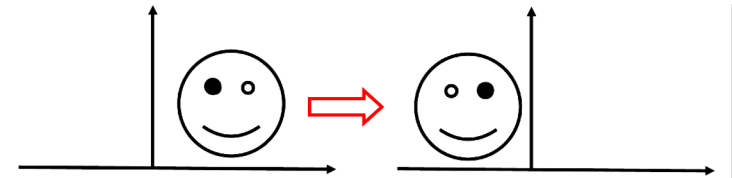
rotation



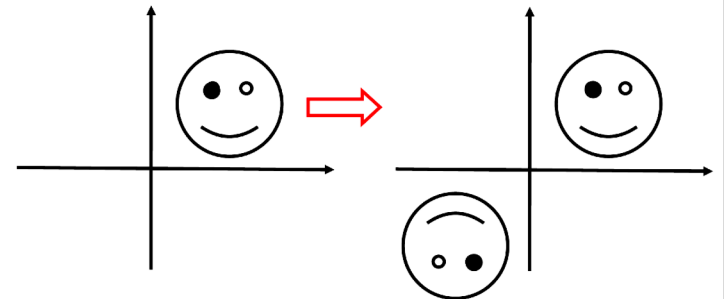
scaling



shear



reflection with respect to the y-axis



reflection with respect to the origin

Translation

- Move (translate, displace) a point to a new location
- Displacement determined by a vector d

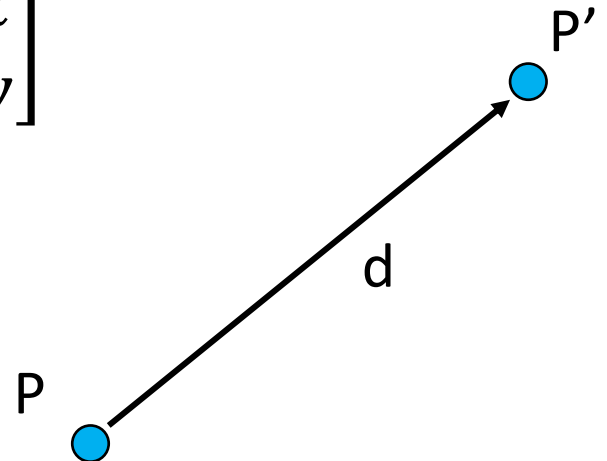
- $P' = P + d$

- $P = \begin{bmatrix} x \\ y \end{bmatrix}, P' = \begin{bmatrix} x' \\ y' \end{bmatrix}, d = \begin{bmatrix} dx \\ dy \end{bmatrix}$

- Simplest transformation:

- $T(v) = v + u$

- Inverse: $T^{-1}(v) = v - u$



Linear Transformations

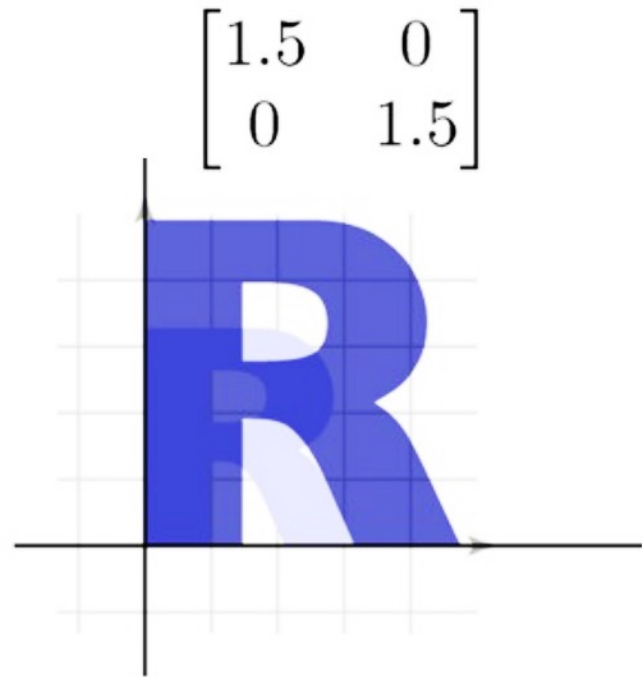
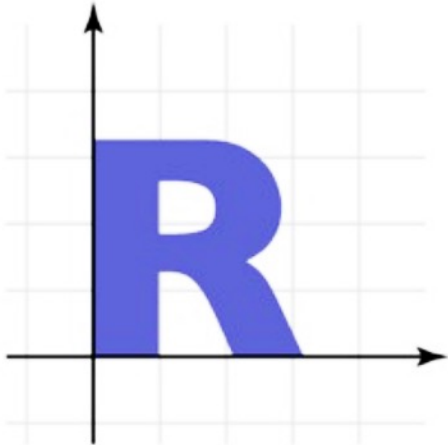
- Such transformations are *linear*:
 - $T(v + u) = T(v) + T(u)$
 - $T(\alpha v) = \alpha T(v)$
 - $T(\alpha v + \beta u) = \alpha T(v) + \beta T(u)$
- Linear Transformations can be represented in *matrix* form
- Rotation, scale, shear, reflection are all *linear*
- Is translation *linear*?

2D Linear Transformations

- $T(v) = Mv = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}_v$
- M is a 2x2 transformation matrix
 - Uniform scale
 - Non-uniform scale
 - Rotation
 - Shear
 - Reflection

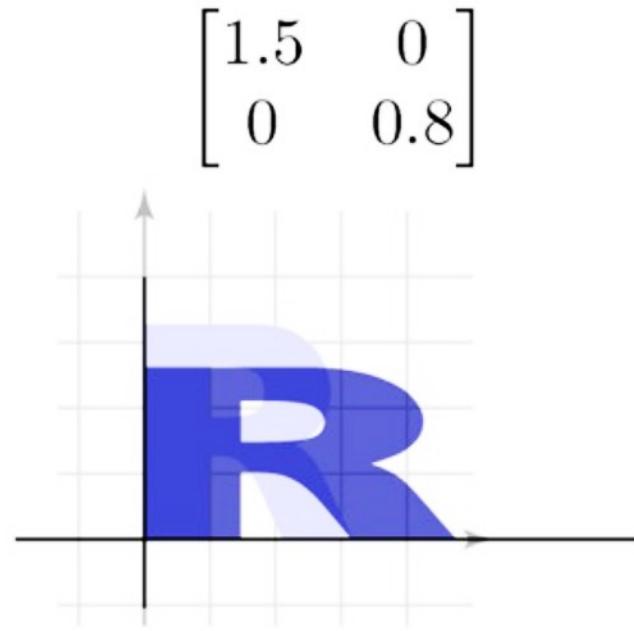
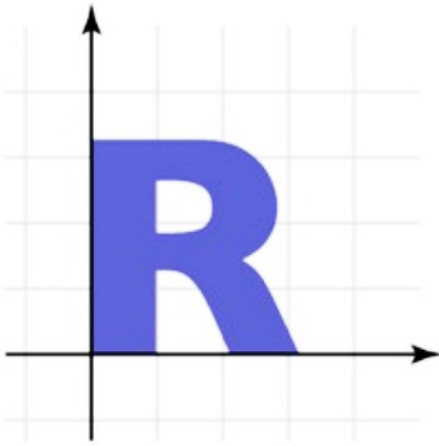
Uniform Scale

- $\begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} sx \\ sy \end{bmatrix}$



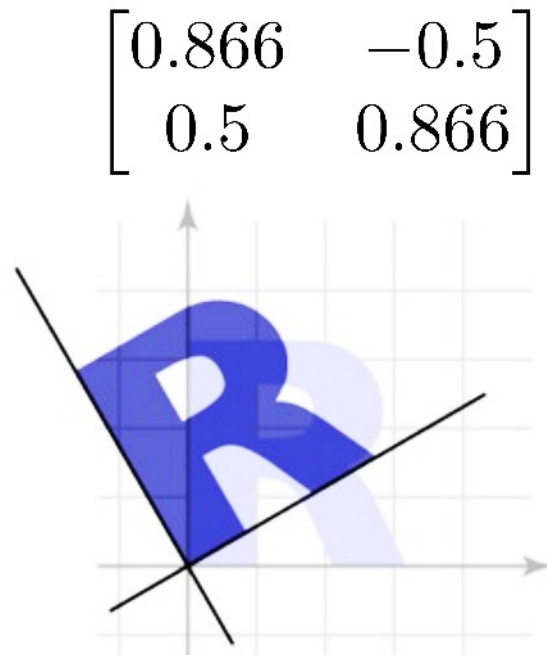
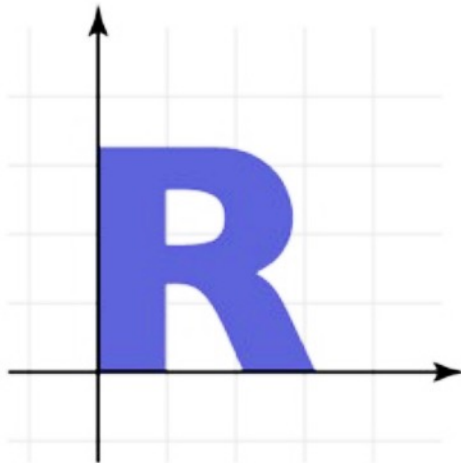
Non-uniform Scale

$$\bullet \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} sx \times x \\ sy \times y \end{bmatrix}$$



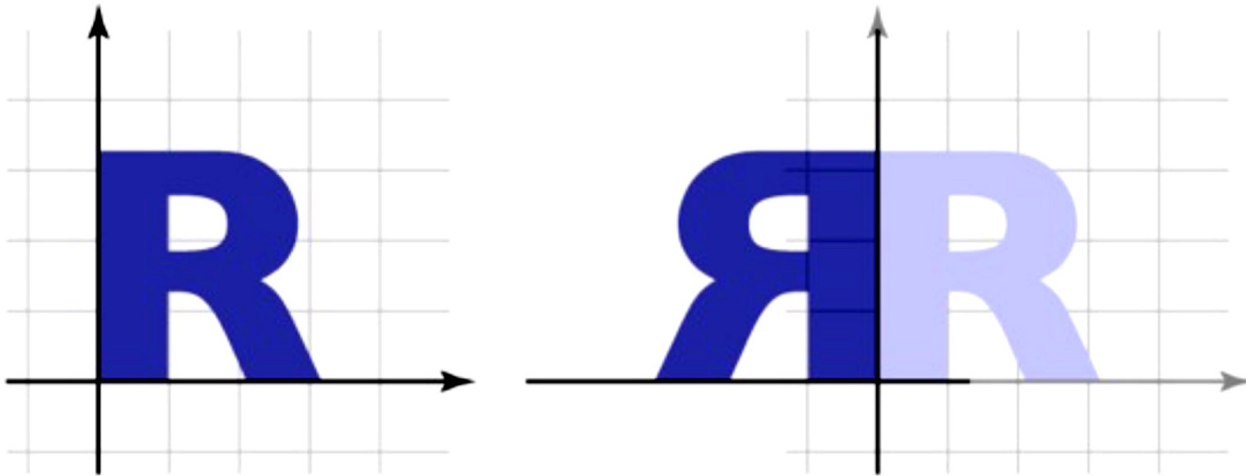
Rotation

$$\bullet \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \end{bmatrix}$$



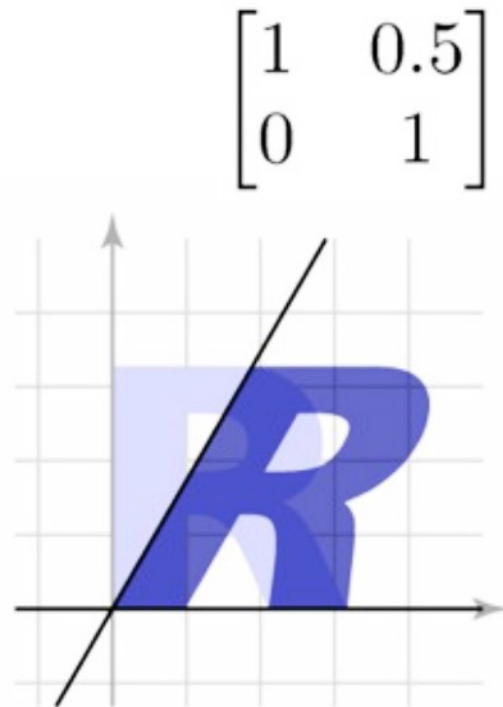
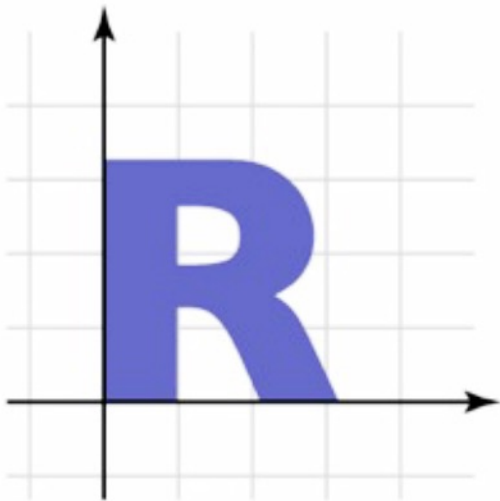
Reflection

- Can be considered as a special case of non-uniform scale
- $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$



Shear

$$\bullet \begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + \alpha y \\ y \end{bmatrix}$$

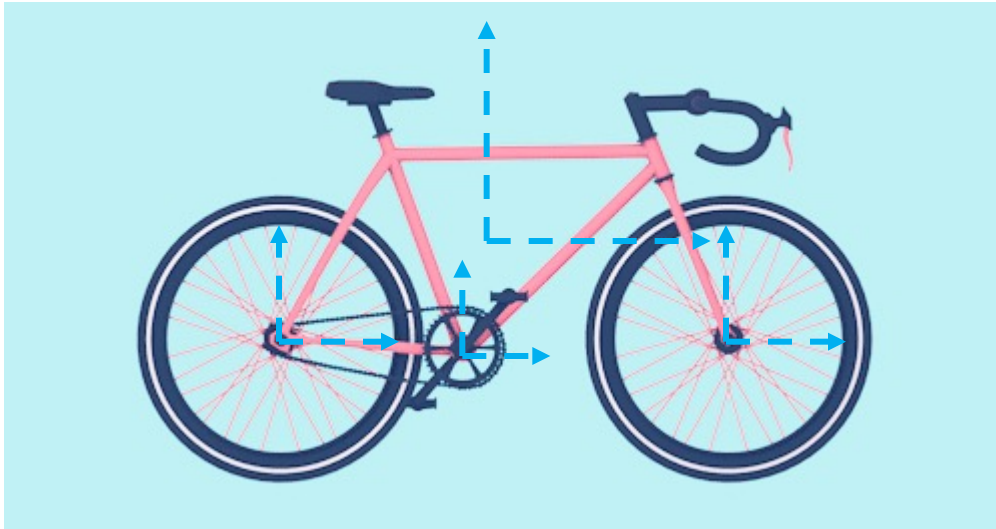


- Questions?

How to animate a bicycle?



How to animate a bicycle?



Composing transformations

- We “move” an object twice or more
 - $P \rightarrow T(P) = P'$
 - $P' \rightarrow S(P') = P''$
 - $P'' = S(P') = S(T(P)) = (S \circ T)(P)$
- $S \circ T$ is S compose T
- **(which one is firstly applied?)**
- Translations
 - $T(P) = P + d_T$
 - $S(P) = P + d_S$
 - $(S \circ T)(P) = P + (d_T + d_S)$
 - Therefore, translation by d_T and d_S is translation by $d_T + d_S$
 - Commutative!

Composing transformations

- Linear Transformations
 - $P \rightarrow T(P) = M_T P = P'$
 - $P' \rightarrow S(P') = M_S P' = P''$
 - $(S \circ T)(P) = M_S M_T P$
 - **(which one is firstly applied?)**
- *Be careful!*
 - *Only some are commutative*

Composing transformations

- Translation together with linear transformations - **Affine transformation**
- Applications require a single framework
 - $T(P) = M_T P + d_T$
 - $S(P) = M_S P + d_S$
 - $(S \circ T)(P) = M_S(M_T P + d_T) + d_S = M_S M_T P + M_S d_T + d_S$
 - Not elegant!

Homogeneous coordinates

- Extra component w
 - $V = [x, y, w]^T$
- If $w \neq 0$, divide by it to get Cartesian coordinates of point $(x/w, y/w, 1)$
 - $P = [x, y, 1]^T$
 - $P' = [x', y', 1]^T$
 - $d = [dx, dy, 0]^T$
- Hence
 - $\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} + \begin{bmatrix} dx \\ dy \\ 0 \end{bmatrix}$

note that this expression is in
Three dimensions and expresses
point = point + vector
Point – point = ?
Can we translate vectors?

Translation Matrix

We can now express translation using a 3 x 3 matrix $\mathbf{T}_d$ in homogeneous coordinates $\mathbf{p}' = \mathbf{T}_d \mathbf{p}$, where

$$\mathbf{T}_d = \mathbf{T}_d(dx, dy) = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}$$

Properties of translations

- $T(0,0) = I$
- $T(s_x, s_y) \cdot T(t_x, t_y) = T(s_x + t_x, s_y + t_y)$
- $T(s_x, s_y) \cdot T(t_x, t_y) = T(t_x, t_y) \cdot T(s_x, s_y)$
- $T^{-1}(s_x, s_y) = T(-s_x, -s_y)$
- Note : translation matrices are *commutative*.

Rotation Matrix

- With Cartesian coordinate frame

- $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$

- With homogenous coordinate frame

$$R = R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \\ 1 \end{bmatrix}$$

Properties of rotation

- Rotation matrices are orthogonal, i.e.:
 - $R^{-1}(\theta) = R^T(\theta)$
- $R^{-1}(\theta) = R(-\theta)$
- $R(0) = I$
- $\text{Det}(R) = 1$
- $R(\theta) \cdot R(\phi) = R(\theta + \phi)$
- And
- $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$
 - But this is only valid when the axis of rotation is the same
 - Not the case in 3D!

Composing transformations

- Compose translation and linear transformations using homogenous coordinates

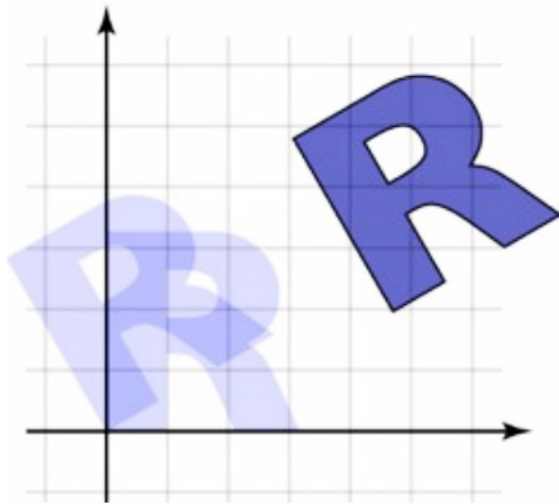
- $$\begin{bmatrix} M_S & d_S \\ 0 & 1 \end{bmatrix} \begin{bmatrix} M_T & d_T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} P \\ 1 \end{bmatrix} = \begin{bmatrix} M_S M_T P + M_S d_T + d_S \\ 1 \end{bmatrix}$$

- This form is better for implementation because all **Affine Transformations** can be expressed this way and multiple transformations can be concatenated together

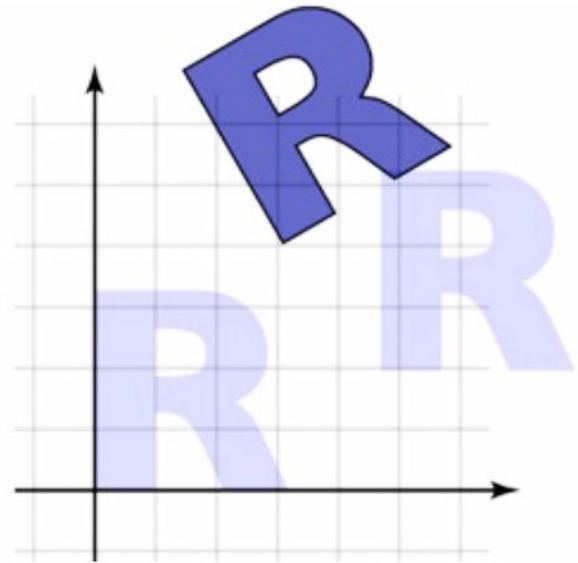
- Questions?

Composite Transformations

- In general NOT commutative

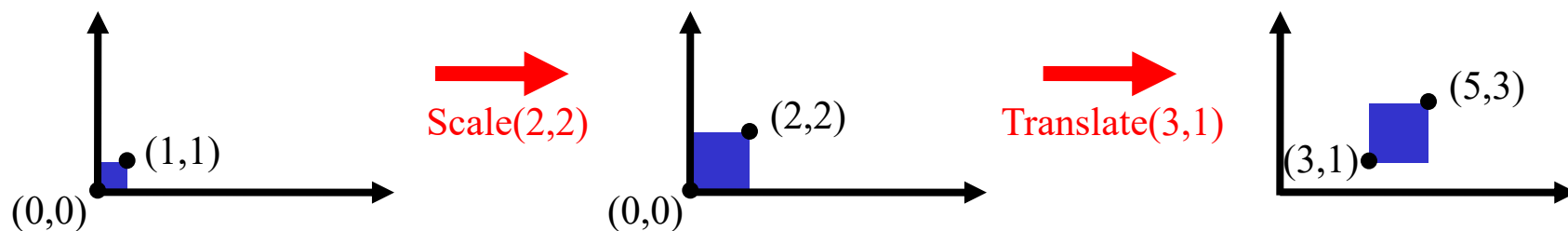


rotate, then translate

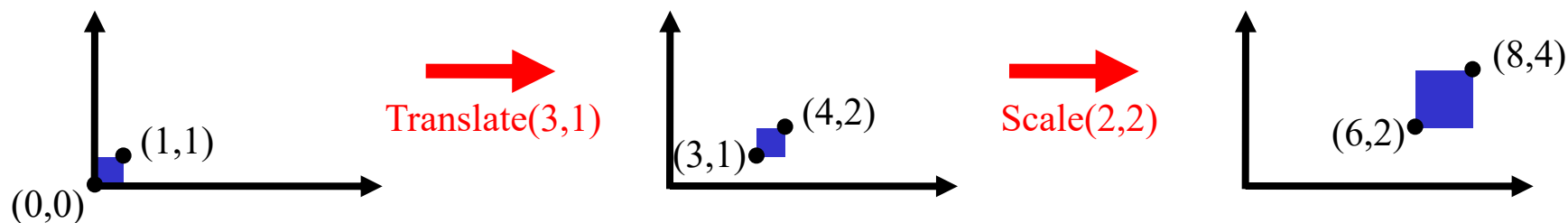


translate, then rotate

Scale then Translate: $p' = T (S p) = TS p$



Translate then Scale: $p' = S (T p) = ST p$



Scale then Translate: $\mathbf{p}' = \mathbf{T} (\mathbf{S} \mathbf{p}) = \mathbf{TS} \mathbf{p}$

$$TS = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Translate then Scale: $\mathbf{p}' = \mathbf{S} (\mathbf{T} \mathbf{p}) = \mathbf{ST} \mathbf{p}$

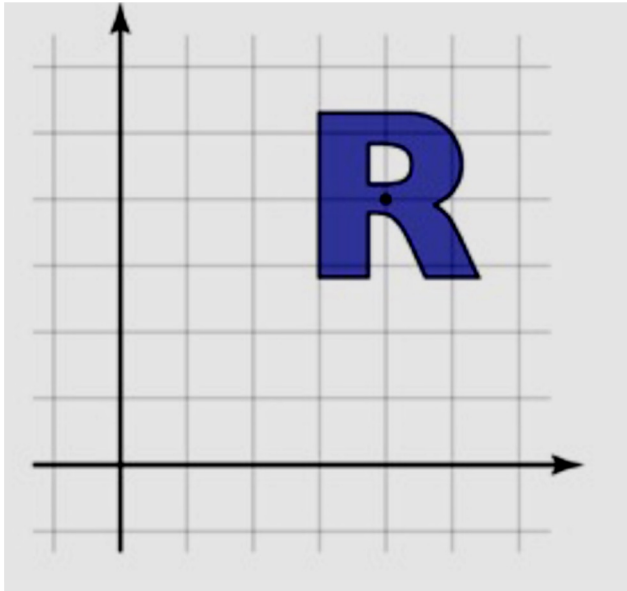
$$ST = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 6 \\ 0 & 2 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$

Properties of Affine transformations

- Rotation and translation
 - Angles and distances are preserved
 - Unit cube is always unit cube
 - ***Rigid-Body*** transformations or ***rigid motion***
- Rotation, translation and scale.
 - Angles & distances not preserved.
 - But parallel lines are.

Want to rotate about a particular point

- We know how to rotate about the origin
 - So translate that point to the origin

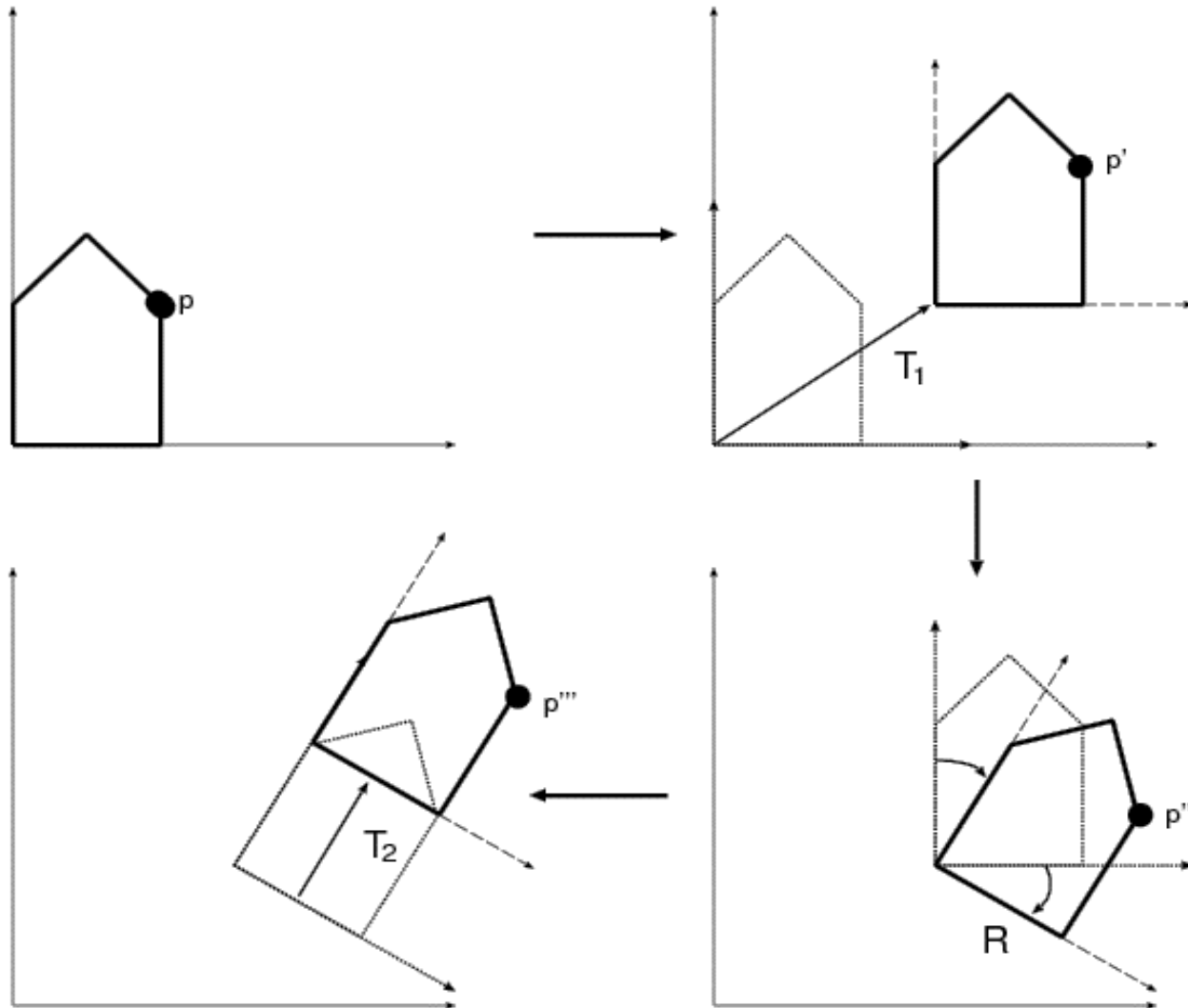


$$M = T^{-1}RT$$

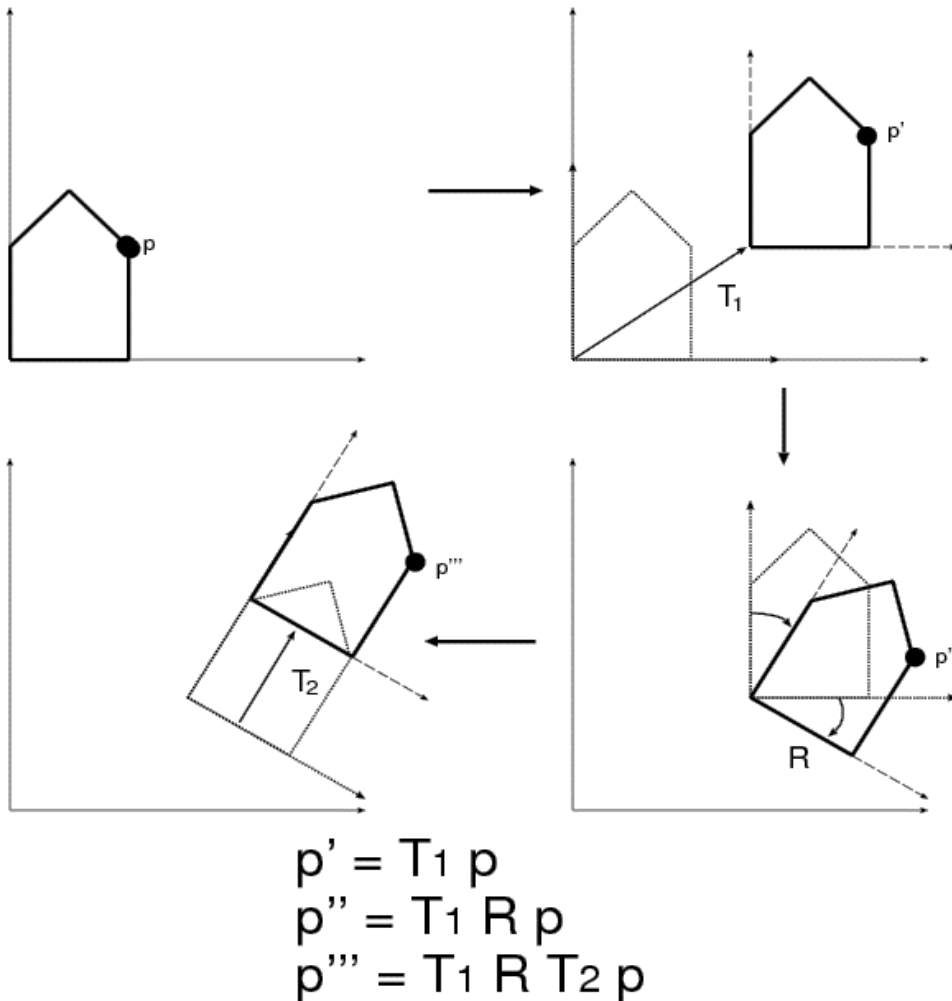
Transformations of local coordinate systems

- Have been discussing transformations as transforming points
- Always need to think the transformation in the world coordinate system
- Sometimes this might not be convenient
 - i.e. rotating objects at its location

Transformations of local coordinate systems - Example



Transformations of local coordinate systems - Example

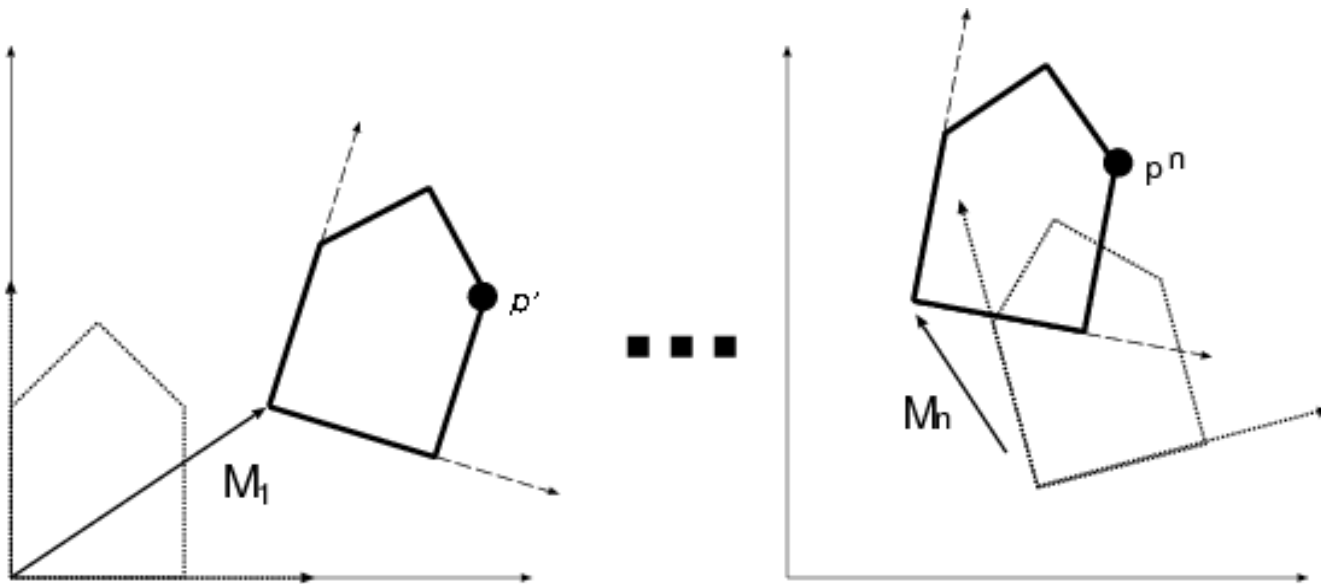


- **Concatenate local transformation matrices from left to right**
- Can obtain the local – world transformation matrix
- p' , p'' , p''' are the world coordinates of p after each transformation

Transformations of local coordinate systems - example

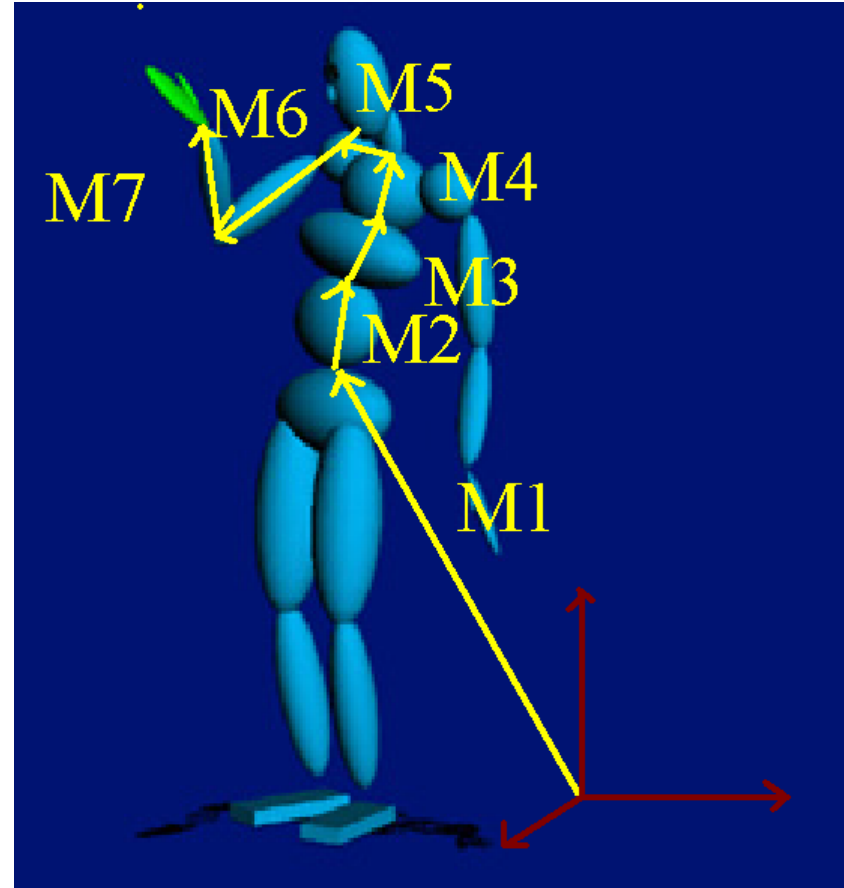
- p^n is the world coordinate of point p after n transformations

$$p^n = M_1 M_2 \cdots M_n p$$



This is convenient for character animation / robotics

- In robotics / animation, we often want to know what is the current 3D location of the end effectors (like the hand)
- Can concatenate matrices from the origin of the body towards the end effector



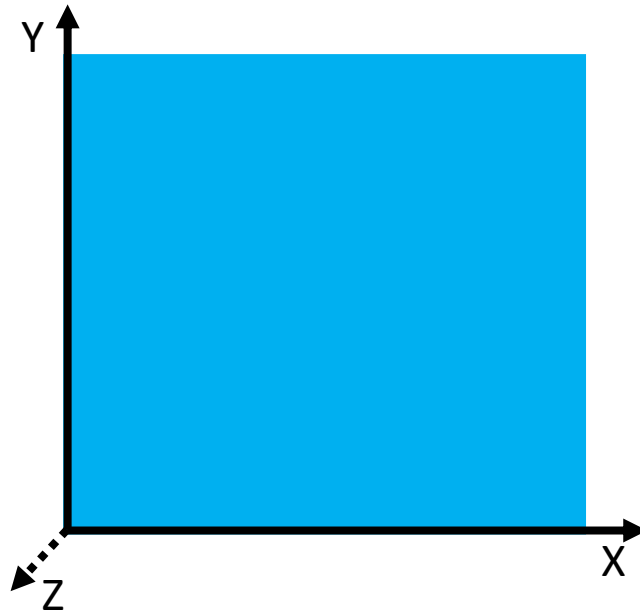
How to implement?

- Queue?
- Stack?
- Tree?
- How to calculate the transformation?

- Questions?

3D Transformations

- Use homogeneous coordinates, just as in 2D case
- Transformations are now 4x4 matrices
- We will use a right-handed (world) coordinate system - (z out of the page)



Translation

$$\bullet \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Scale

$$\bullet \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Reflection

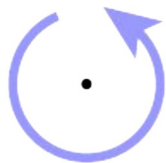
$$\bullet \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_1 & 0 & 0 & 0 \\ 0 & s_2 & 0 & 0 \\ 0 & 0 & s_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Shear

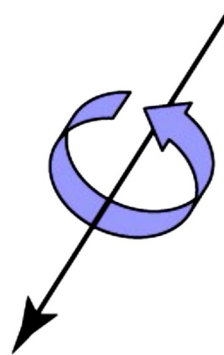
$$\bullet \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & a_1 & a_2 & 0 \\ a_3 & 1 & a_4 & 0 \\ a_5 & a_6 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

General Rotation Matrices

- Rotation in 2D?
 - Around an arbitrary point
- Rotation in 3D?
 - Around an arbitrary axis
 - There are many more 3D rotations than 2D



2D



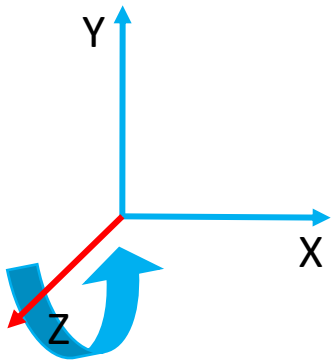
3D

Specifying Rotations

- In 2D, just an angle θ
- In 3D, is more complex
 - Basic rotation about origin: Axis and angle
 - Convention: positive rotation is CCW (right handed)
 - Many ways
 - Directly through
 - Euler angles: 3 angles about 3 axes
 - Indirectly through frame transformations
 - Quaternions

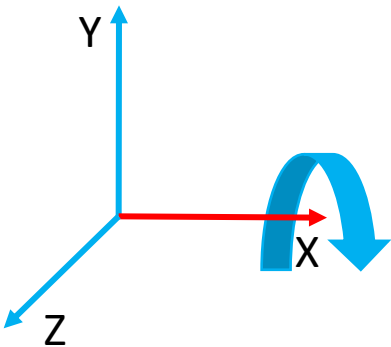
Rotation

$$\bullet R_z(\theta) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



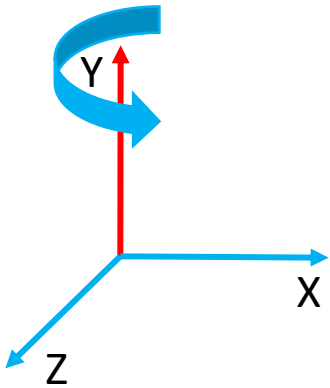
Rotation

$$\bullet R_x(\theta) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



Rotation

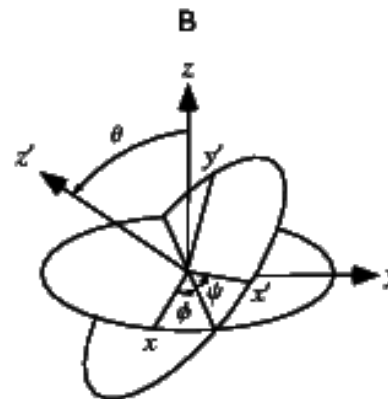
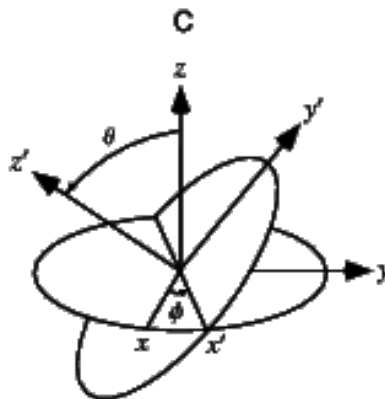
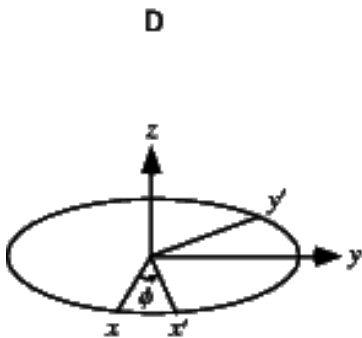
$$\bullet R_y(\theta) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



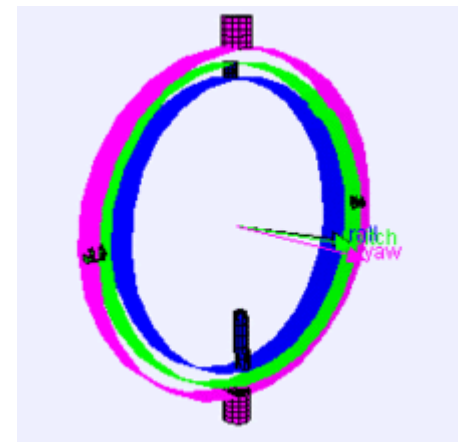
Euler Angles

- Any rotation may be described using three angles. If the rotations are written in terms of rotation matrices D, C, and B, then a general rotation A can be written as:

$$A = BCD$$

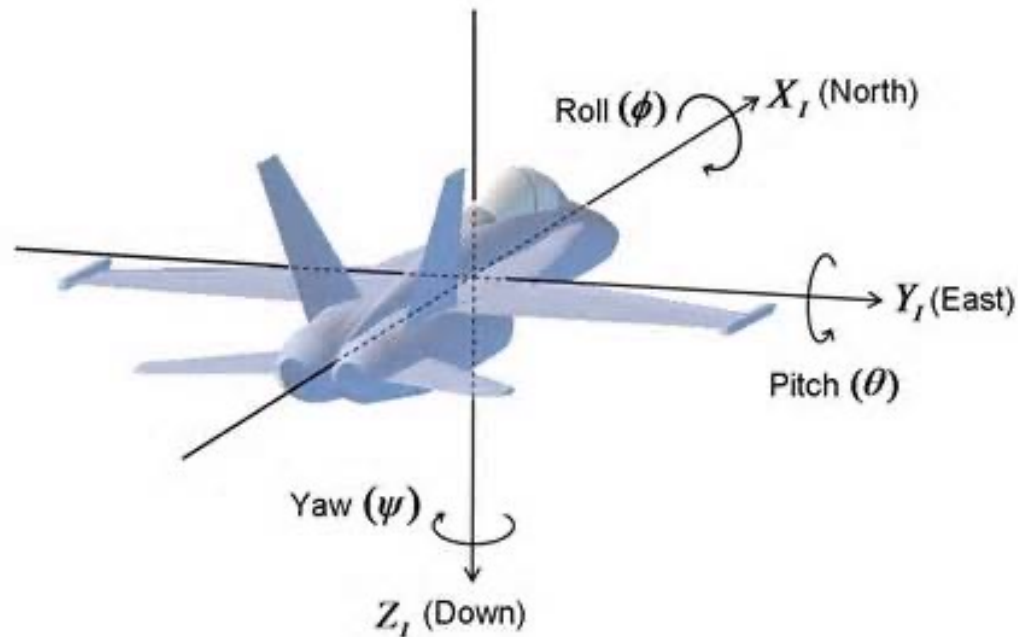


Gimbal



Euler Angles

- Roll(Φ) around X
- Pitch(Θ) around Y
- Yaw(Ψ) around Z

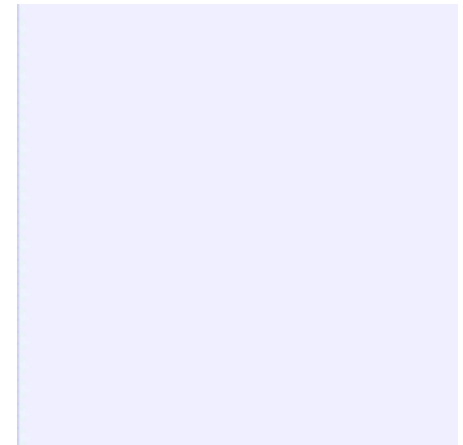
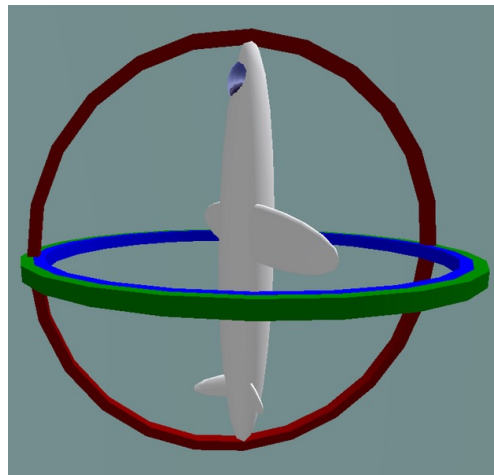
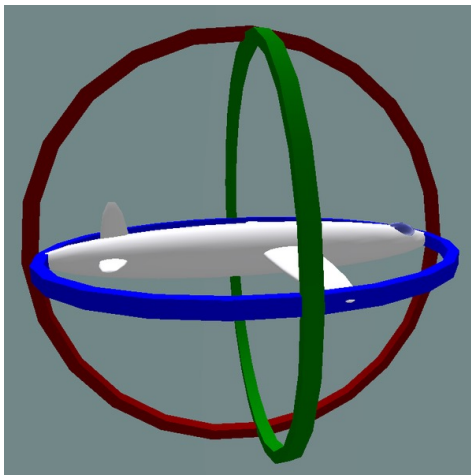


Gimbal Lock

- Euler basic motions are never expressed in a frame, but a **mixed axes of rotation**

- $$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & 0 & -\sin\beta \\ 0 & 1 & 0 \\ \sin\beta & 0 & \cos\beta \end{bmatrix} \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Set $\beta = \pi/2$?

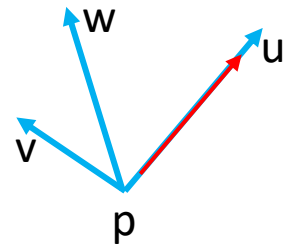
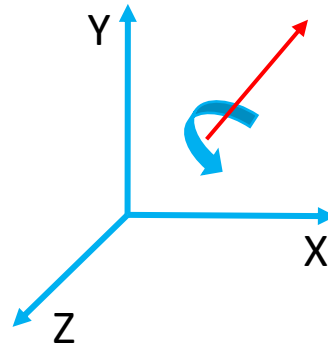


Matrices for Axis-angle Rotations

- Ruler angles are for coordinates axes
 - What if we want rotation about some random axis?
- Compute by composing elementary transforms
 - Transform rotation axis to align with x axis
 - Apply rotation
 - Inverse transform back into position

Building General Rotations

- Using elementary transforms
 - Translate axis to pass through origin
 - Rotate about y to get into x-y plane
 - Rotate about z to get into x axis
- Alternative: construct a frame and change coordinates
 - Choose p, u, v, w
 - Apply transform $T = FR_x(\theta)F^{-1}$
 - $F = \begin{bmatrix} u & v & w & p \\ 0 & 0 & 0 & 1 \end{bmatrix}$



Quaternions

- Compare to Euler angles and Rotation matrices
 - Can avoid the gimbal lock
 - Can smoothly interpolate over a sphere
- Representation
 - A quaternion is composed of one real element and three complex elements

$$q = q_0 + q_1i + q_2j + q_3k$$

$$\begin{cases} i^2 = j^2 = k^2 = -1 \\ ij = k, ji = -k \\ jk = i, kj = -i \\ ki = j, ik = -j \end{cases}$$

Quaternions

$$\mathbb{H} := \text{span}(\{1, i, j, k\})$$

$$q = a + bi + cj + dk \in \mathbb{H}$$



William Rowan Hamilton
(1805-1865)



(Not Hamilton)

Here as he walked by
on the 16th of October 1843
Sir William Rowan Hamilton
in a flash of genius discovered
the fundamental formula for
quaternion multiplication
 $i^2 = j^2 = k^2 = ijk = -1$
& cut it on a stone of this bridge

Quaternions

- A rotation through an angle of θ around the axis defined by a unit vector $n = [n_x, n_y, n_z]^T$ can be represented by a quaternion:

$$q = \left[\cos \frac{\theta}{2}, n_x \sin \frac{\theta}{2}, n_y \sin \frac{\theta}{2}, n_z \sin \frac{\theta}{2} \right]^T$$

- Rotation: $p' = qpq^{-1}$
- Rotation matrix, Rotation Axis, Euler Angle and Quaternions are exchangeable. [Quaternions and spatial rotation - Wikipedia](#)

Rodriguez formula

$$R = \cos(\theta) \cdot I + (1 - \cos(\theta))\phi\phi^T + \sin(\theta)\phi_{\times}$$

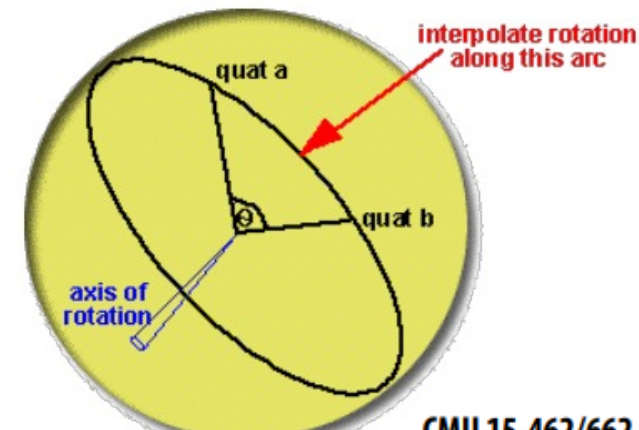
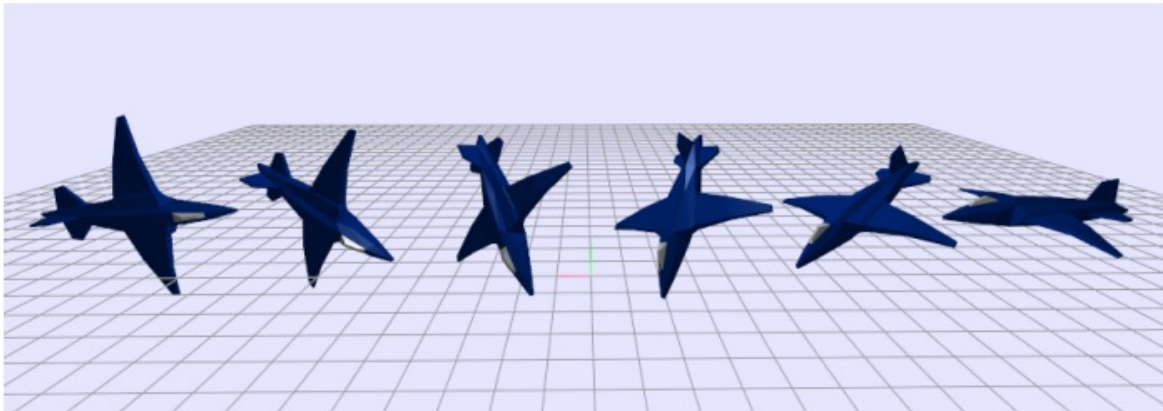
- I identity matrix, ϕ **axis-angle rotation**
- $\times$ is skew-symmetric matrix of a vector:

$$(\mathbf{a} \times) = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}$$

Interpolating Rotations

- Suppose we want to smoothly interpolate between two rotations (e.g., orientations of an airplane)
- Interpolating Euler angles can yield strange-looking paths, non-uniform rotation speed, ...
- Simple solution* w/ quaternions: “SLERP” (spherical linear interpolation)

$$\text{Slerp}(q_0, q_1, t) = q_0(q_0^{-1}q_1)^t, \quad t \in [0, 1]$$



Transformations with GLM

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include <glm/gtx/quaternion.hpp>

//rotation matrix
glm::vec4 vec(1.0f, 0.0f, 0.0f, 1.0f);
glm::mat4 trans = glm::mat4(1.0f);
trans = glm::translate(trans, glm::vec3(1.0f, 1.0f, 0.0f));
trans = glm::rotate(trans, glm::radians(90.0f), glm::vec3(0.0, 0.0, 1.0));
vec = trans * vec;
...
//quaternions
glm::quat rot = glm::angleAxis(glm::radians(90.f), glm::vec3(0.f, 1.f, 0.f));
glm::mat4 rotate = glm::mat4_cast(rot)
```

- Questions?

References

- Ed Angel, CS/EECE 433 Computer Graphics, University of New Mexico
- Durand and Cuter, MIT EECS 6.837
- Steve Marschner, CS4620/5620 Computer Graphics, Cornell
- Tom Thorne, COMPUTER GRAPHICS, The University of Edinburgh
- Elif Tosun, Computer Graphics, The University of New York
- CMU 15/462/662 Computer Graphics, Carnegie Mellon University